

A view of 0 or 1 elements: `view::maybe`

- Document number: P1255R3
- Date: 2019-03-10
- Author: Steve Downey <sdowney2@bloomberg.net>, <sdowney@gmail.com>
- Audience: LEWG

Abstract: This paper proposes `view::maybe` a range adaptor that produces a view with cardinality 0 or 1 which adapts nullable types such as `std::optional` and pointer to object types.

Table of Contents

- [1. Changes](#)
- [2. Motivation](#)
- [3. Proposal](#)
- [4. Design](#)
- [5. Synopsis](#)
- [6. Impact on the standard](#)
- [7. References](#)
- [8. Appendix A : `view\_maybe` - deep const version](#)

1 Changes

1.1 Changes since R2

1.1.1 Reflects current code as reviewed

1.1.2 Nullable concept specification

Remove `Readable` as part of the specification, use the useful requirements from `Readable`

1.1.3 Wording for `view::maybe` as proposed

1.1.4 Appendix A: wording for a `view_maybe` that always captures

1.2 Changes since R1

1.2.1 Refer to `view::all`

Behavior of capture vs refer is similar to how `view::all` works over the expression it is given

1.2.2 Use wording 'range adaptor object'

Match current working paper language

1.3 Changes since R0

1.3.1 Remove customization point objects

Removed `view::maybe_has_value` and `view::maybe_value`, instead requiring that the nullable type be dereferenceable and contextually convertible to `bool`.

1.3.2 Concept `Nullable`, for exposition

Concept `Nullable`, which is `Readable` and contextually convertible to `bool`

1.3.3 Capture rvalues by decay copy

Hold a copy when constructing a view over a nullable rvalue.

1.3.4 Remove `maybe_view` as a specified type

Introduced two exposition types, one safely holding a copy, the other referring to the nullable

2 Motivation

In writing range transformation pipelines it is useful to be able to lift a nullable value into a view that is either empty or contains the value held by the nullable. The adapter `view::single` fills a similar purpose for non-nullable values, lifting a single value into a view, and `view::empty` provides a range of no values of a given type. A `view::maybe` adaptor also allows nullable values to be treated as ranges when it is otherwise undesirable to make them containers, for example `std::optional`.

```
std::vector<std::optional<int>> v{
    std::optional<int>{42},
    std::optional<int>{},
    std::optional<int>{6 * 9}};

auto r = view::join(view::transform(v, view::maybe));

for (auto i : r) {
    std::cout << i; // prints 42 and 54
}
```

In addition to range transformation pipelines, `view::maybe` can be used in range based for loops, allowing the nullable value to not be dereferenced within the body. This is of small value in small examples in contrast to testing the nullable in an `if` statement, but with longer bodies the dereference is often far away from the test. Often the first line in the body of the `if` is naming the dereferenced nullable, and lifting the dereference into the for loop eliminates some boilerplate code, the same way that range based for loops do.

```
{
    auto&& opt = possible_value();
    if (opt) {
        // a few dozen lines ...
        use(*opt); // is *opt OK ?
    }
}

for (auto&& opt : view::maybe(possible_value())) {
    // a few dozen lines ...
    use(opt); // opt is OK
}
```

The design permits writing through the view, also:

```
std::optional o{7};
for (auto&& i : view::maybe(o)) {
    i = 9;
    std::cout << "i=" << i << " prints 9\n";
}
std::cout << "o=" << *o << " prints 9\n";

auto oe = std::optional<int>{};
for (int i : view::maybe(oe))
    std::cout << "i=" << i << '\n'; // does not print
```

3 Proposal

Add a range adaptor object `view::maybe`, returning a view over a nullable object, capturing by value temporary nullables. A `Nullable` object is one that is both contextually convertible to `bool` and for which the type produced by dereferencing is an equality preserving object. Non void pointers, `std::optional`, and the proposed outcome and expected types all model `Nullable`. Function pointers do not, as functions are not objects. Iterators do not generally model `Nullable`, as they are not required to be contextually convertible to `bool`.

4 Design

The basis of the design is to hybridize `view::single` and `view::empty`. If the underlying object claims to hold a value, as determined by checking if the object when converted to `bool` is true, begin and end of the view are equivalent to the address of the held value within the underlying object and one past the underlying object. If the underlying object does not have a value, begin and end return `nullptr`.

The `view::maybe` range adapter object will create either a safe view, containing a move initialized `decay_copy` of the nullable, or a reference view, referring to the nullable value, depending on the deduced referenceness of the template parameter. This is similar to `view::all`, without attempting to handle its third case defaulting to using `subrange`, as there seems to be no good model of an expression that falls into that third category.

5 Synopsis

5.1 Maybe View

`view::maybe` returns a `View` over a `Nullable` that is either empty if the nullable is empty, or provides access to the contents of the nullable object.

The name `view::maybe` denotes a range adaptor object (`[range.adaptor.object]`). For some subexpression `E`, the expression `view::maybe(E)` is expression-equivalent to:

- `safe_maybe_view{E}`, the exposition only `View` specified below, if the expression is well formed, where `decay-copy(E)` is moved into the `safe_maybe_view`
- otherwise `ref_maybe_view{E}`, the exposition only `View` specified below, if that expression is well formed, where `ref_maybe_view` refers to `E`
- otherwise `view::maybe(E)` is ill-formed.

Note: Whenever `view::maybe(E)` is a valid expression, it is a `pvalue` whose type models `View`. — end note]

5.2 Concept *Nullable*

Types that:

- are contextually convertible to bool
- are dereferenceable
- have const references which are dereferenceable
- the `iter_reference_t` of the type and the `iter_reference_t` of the const type, will :
- satisfy `is_lvalue_reference`
- satisfy `if_object` when the reference is removed
- for const pointers to the referred to types, satisfy `ConvertibleTo` model the exposition only `Nullable` concept

Given a value `i` of type `I`, `I` models `Nullable` only if the expression `*i` is equality-preserving. [Note: The expression `*i` is indirectly required to be valid via the exposition-only `dereferenceable` concept ([iterator.synopsis]). — end note]

```
namespace std::ranges {  
    // For Exposition  
    template <class T, class Ref, class ConstRef>  
    concept bool _ReadableReferences =  
        is_lvalue_reference_v<Ref> &&  
        is_object_v<remove_reference_t<Ref>> &&  
        is_lvalue_reference_v<ConstRef> &&  
        is_object_v<remove_reference_t<ConstRef>> &&  
        ConvertibleTo<add_pointer_t<ConstRef>,  
            const remove_reference_t<Ref>*>;  
  
    template <class T>  
    concept bool Nullable =  
        is_object_v<T> &&  
        requires(T& t, const T& ct) {  
            bool(ct); // Contextually bool  
            *t; // T& is dereferenceable  
            *ct; // const T& is dereferenceable  
        }  
    && _ReadableReferences<T,  
        iter_reference_t<T>, // Ref  
        iter_reference_t<const T>>; // ConstRef
```

5.3 *safe_maybe_view*

```
// For Exposition  
template <Nullable Maybe>  
requires ranges::CopyConstructible<Maybe>  
class safe_maybe_view  
    : public ranges::view_interface<safe_maybe_view<Maybe>> {  
private:  
    using T = remove_reference_t<ranges::iter_reference_t<Maybe>>;  
  
    semiregular_box<Maybe> value_;
```

```

public:
    constexpr safe_maybe_view() = default;
    constexpr explicit safe_maybe_view(Maybe const& maybe) noexcept(
        std::is_nothrow_copy_constructible_v<Maybe>)
        : value_(maybe) {}
    constexpr explicit safe_maybe_view(Maybe&& maybe) noexcept(
        std::is_nothrow_move_constructible_v<Maybe>)
        : value_(std::move(maybe)) {}

    constexpr T* begin() noexcept { return data(); }
    constexpr const T* begin() const noexcept { return data(); }
    constexpr T* end() noexcept { return data() + size(); }
    constexpr const T* end() const noexcept { return data() + size(); }

    constexpr std::ptrdiff_t size() const noexcept {
        return bool(value_.get());
    }

    constexpr T* data() noexcept {
        Maybe& m = value_.get();
        return m ? std::addressof(*m) : nullptr;
    }

    constexpr const T* data() const noexcept {
        Maybe const& m = value_.get();
        return m ? std::addressof(*m) : nullptr;
    }
};

```

5.4 ref_maybe_view

```

// For Exposition
template <Nullable Maybe>
class ref_maybe_view
    : public ranges::view_interface<ref_maybe_view<Maybe>> {
    using T = remove_reference_t<ranges::iter_reference_t<Maybe>>;

    Maybe* value_ = nullptr;

public:
    constexpr ref_maybe_view() = default;
    constexpr explicit ref_maybe_view(Maybe& maybe) noexcept
        : value_(std::addressof(maybe)) {}

    constexpr T* begin() noexcept { return data(); }
    constexpr const T* begin() const noexcept { return data(); }
    constexpr T* end() noexcept { return data() + size(); }
    constexpr const T* end() const noexcept { return data() + size(); }

    constexpr std::ptrdiff_t size() const noexcept { return bool(*value_); }

    constexpr T* data() noexcept {
        return *value_ ? std::addressof(**value_) : nullptr;
    }
    constexpr const T* data() const noexcept {
        return *value_ ? std::addressof(**value_) : nullptr;
    }
};

```

```

// For Exposition
namespace view {
struct __maybe_fn {
    template <Nullable T>
    constexpr ref_maybe_view<T> operator()(T& t) const
        noexcept {
        return ref_maybe_view<T>{t};
    }

    template <class T,
              Nullable U = remove_cv_t<T>>
    requires ranges::Constructible<U, T> &&
             ranges::CopyConstructible<U>
    constexpr safe_maybe_view<U> operator()(T&& t) const
        noexcept(is_nothrow_constructible_v<U, T>) {
        return safe_maybe_view<U>{move(t)};
    }
};

inline constexpr __maybe_fn maybe{};

} // namespace view
} // namespace std::ranges

```

[Example:

```

optional o{4};
for (int i : view::maybe(o))
    cout << i; // prints 4

maybe_view e{ };
for (int i : view::maybe(optional{}))
    cout << i; // does not print

int      j  = 8;
int*     pj = &j;
for (auto i : view::maybe(pj))
    std::cout << i; // prints 8

typedef int (*func)(int);
func f = nullptr;
maybe_view{f}; // Error, a function is not an object, so does not satisfy Nullable

```

— end example]

6 Impact on the standard

A pure library extension, affecting no other parts of the library or language.

7 References

[P0896R3] Eric Niebler, Casey Carter, Christopher Di Bella. The One Ranges Proposal URL: <https://wg21.link/p0896r3>

[P0323R7] Vicente Botet, JF Bastien. std::expected URL: <https://wg21.link/p0323r7>

8 Appendix A : view_maybe - deep const version

8.1 Maybe view

8.1.1 Overview

maybe_view produces a View that contains either zero or one element of a specified value.

[Example:

```
maybe_view o{4};
for (int i : o)
    cout << i; // prints 4

maybe_view z{};
for (int i : z)
    cout << i; // does not print
```

— end example]

8.1.2 Class template maybe_view

```
namespace std::ranges {
template <Nullable Maybe>
    requires CopyConstructible<Maybe>
class maybe_view
    : public view_interface<maybe_view<Maybe>> {
private:
    // For Exposition
    using T = remove_reference_t<iter_reference_t<Maybe>>;
    semiregularbox<Maybe> value_;

public:
    constexpr maybe_view() = default;
    constexpr explicit maybe_view(Maybe const& maybe)
        noexcept(std::is_nothrow_copy_constructible_v<Maybe>);

    constexpr explicit maybe_view(Maybe&& maybe)
        noexcept(std::is_nothrow_move_constructible_v<Maybe>);

    template<class... Args>
    requires Constructible<Maybe, Args...>
    constexpr maybe_view(in_place_t, Args&&... args);

    constexpr T* begin() noexcept;
    constexpr const T* begin() const noexcept;
    constexpr T* end() noexcept;
    constexpr const T* end() const noexcept;

    constexpr std::ptrdiff_t size() const noexcept;

    constexpr T* data() noexcept;
    constexpr const T* data() const noexcept;
};

constexpr explicit maybe_view(const Maybe& maybe);
}
```

```
constexpr explicit maybe_view(Maybe const& maybe)
    noexcept(std::is_nothrow_copy_constructible_v<Maybe>);
```

Effects: Initializes value_ with maybe. [↗](#)

```
constexpr explicit maybe_view(Maybe&& maybe)
    noexcept(std::is_nothrow_move_constructible_v<Maybe>);
```

Effects: Initializes value_ with `std::move(maybe)`. [↗](#)

```
template<class... Args>
constexpr maybe_view(in_place_t, Args&&... args);
```

Effects: Initializes value_ as if by `value_{in_place, std::forward<Args>(args)...}`. [↗](#)

```
constexpr T* begin() noexcept;
constexpr const T* begin() const noexcept;
```

Effects: Equivalent to: `return data();`. [↗](#)

```
constexpr T* end() noexcept;
constexpr const T* end() const noexcept;
```

Effects: Equivalent to: `return data() + size();`. [↗](#)

```
static constexpr ptrdiff_t size() noexcept;
```

Effects: Equivalent to: `return bool(value_.get());`. [↗](#)

```
constexpr T* data() noexcept;
constexpr const T* data() const noexcept;
```

Effects: Equivalent to:

```
Maybe& m = value_.get();
return m ? std::addressof(*m) : nullptr;
```

8.1.3 view::maybe

The name `view::maybe` denotes a customization point object ([customization.point.object]). For some subexpression `E`, the expression `view::maybe(E)` is expression-equivalent to `maybe_view{E}`.